

YouTube Channel Subscription System

Observer Design Pattern Project

■ Introduction

Modern applications often need a mechanism to notify multiple users automatically whenever a specific event occurs. One common real-world example is YouTube, where subscribers receive notifications whenever a channel uploads a new video.

This project demonstrates the implementation of the Observer Design Pattern in Java by creating a simple YouTube subscription system.

In this system:

Users can subscribe to a YouTube channel.

The channel uploads videos.

All subscribers receive notifications automatically when a new video is uploaded.

This creates a flexible communication system between the channel and its subscribers.

■ Design Pattern Used

■ Observer Design Pattern

The Observer Pattern is a behavioral design pattern used to establish a one-to-many relationship between objects.

When the state of one object changes:

all dependent objects are notified automatically.

In this project:

The YouTubeChannel acts as the Subject.

The Subscribers act as the Observers.

■ Why Did We Use Observer Pattern?

We used the Observer Pattern because it perfectly fits systems that require automatic notifications between objects.

In the YouTube subscription system:

many subscribers need to be informed whenever a new video is uploaded,
subscribers can join or leave dynamically,
and the channel should notify everyone without tightly connecting the classes together.

Using the Observer Pattern provides several advantages:

Loose coupling between the channel and subscribers.

Easy addition or removal of subscribers.

Automatic notification mechanism.

Better scalability and maintainability.

This makes the system cleaner, more organized, and easier to expand in the future.

■ Problem Statement

In a YouTube platform, users subscribe to channels to receive updates about newly uploaded videos.

Without using a proper design pattern:

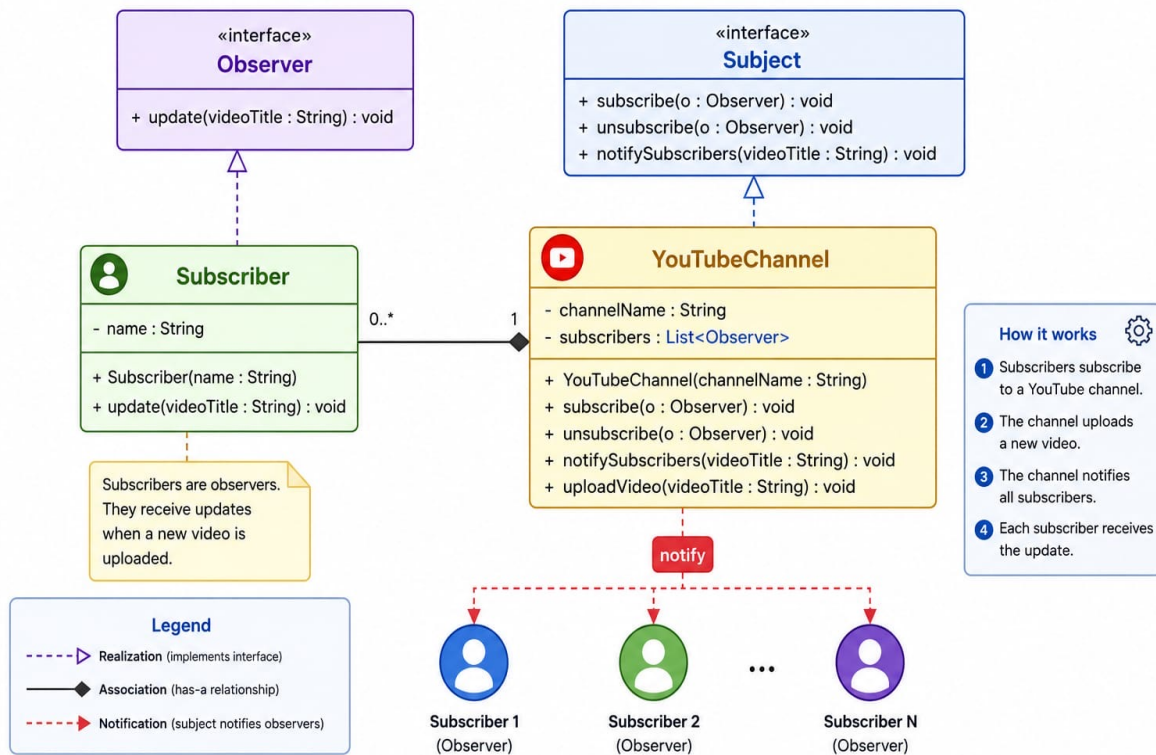
the channel would need to manually notify every subscriber,
the code would become tightly coupled,
and managing subscribers would become difficult as the system grows.

Therefore, we need a flexible solution that allows:

adding subscribers dynamically,
removing subscribers easily,
and notifying all subscribers automatically whenever a new video is uploaded.

The Observer Design Pattern solves this problem efficiently by creating a subscription mechanism between the channel and its subscribers.

YouTube Channel Subscription System (Observer Design Pattern)



```

Java
public interface Observer {
    void update(String videoTitle);
}
    
```

2 ■■ Subject Interface

```

Java
public interface Subject {

    void subscribe(Observer observer);

    void unsubscribe(Observer observer);

    void notifySubscribers(String videoTitle);
}
    
```

3 ■■ Subscriber Class

```

Java
public class Subscriber implements Observer {

    private String name;

    public Subscriber(String name) {
        this.name = name;
    }

    @Override
    public void update(String videoTitle) {

        System.out.println(name +
            " received notification: New video uploaded -> "
            + videoTitle);
    }
}
    
```

4 ■■ YouTubeChannel Class

```

Java
    
```

```

import java.util.ArrayList;
import java.util.List;

public class YouTubeChannel implements Subject {

    private List<Observer> subscribers = new ArrayList<>();
    private String channelName;

    public YouTubeChannel(String channelName) {
        this.channelName = channelName;
    }

    @Override
    public void subscribe(Observer observer) {
        subscribers.add(observer);
    }

    @Override
    public void unsubscribe(Observer observer) {
        subscribers.remove(observer);
    }

    @Override
    public void notifySubscribers(String videoTitle) {
        for (Observer observer : subscribers) {
            observer.update(videoTitle);
        }
    }

    public void uploadVideo(String videoTitle) {
        System.out.println("\n" + channelName
            + " uploaded new video: " + videoTitle);

        notifySubscribers(videoTitle);
    }
}

```

5 ■ Main Class

```

Java
public class Main {

    public static void main(String[] args) {

        YouTubeChannel channel =
            new YouTubeChannel("Programming Hub");

        Subscriber s1 = new Subscriber("Ahmad");
        Subscriber s2 = new Subscriber("Lina");
        Subscriber s3 = new Subscriber("Omar");

        channel.subscribe(s1);
        channel.subscribe(s2);
        channel.subscribe(s3);

        channel.uploadVideo("Observer Pattern in Java");

        channel.unsubscribe(s2);

        channel.uploadVideo("Design Patterns Tutorial");
    }
}

```

■ Expected Output

Plain text

Programming Hub uploaded new video: Observer Pattern in Java

Ahmad received notification: New video uploaded -> Observer Pattern in Java

Lina received notification: New video uploaded -> Observer Pattern in Java

Omar received notification: New video uploaded -> Observer Pattern in Java

Programming Hub uploaded new video: Design Patterns Tutorial

Ahmad received notification: New video uploaded -> Design Patterns Tutorial

Omar received notification: New video uploaded -> Design Patterns Tutorial

■■■■■ ■■■■■